

DATA 260 Homework 5 Report

GITHUB REPOSITORY: <https://github.com/rutujaste06/data260-5825>

Student ID- **020065825**

SID4- **5825**

PORT_BASE - **8425**

PREFIX - **s5825**

SEED - **5825**

VERIFY_SEED – **265825**

DOMAIN_ID - **1**

DOMAIN - Clinical trial listings

HARDWARE - 13th Gen Intel Core i7-13700HX (2.10 GHz), 16GB RAM, NVIDIA GeForce RTX 4070 Laptop GPU (8GB)

LOCAL MODEL USED: **qwen3:8b (Ollama)**

TAGGED COMMIT HASH: **714d9fb7436bf5b5e06349ad43ab117b65cc6355**

Outputs:

B. Domain MCP Server (required)

Search trials will provide the complete information :

The screenshot displays a web browser window with the URL `127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=7494f7c0e4ecb0d4377399dac3005c2a94418a7dbb4f17456893a8f9c367df96`. The browser's address bar shows a Chrome notification: "Continue where you left off: Chrome can restore your tabs every time you restart. To turn this on, go to [Settings](#)."

The interface is titled "trials" and features a navigation bar with tabs for "Servers", "Tools", "Prompts", and "Resources". The "Tools" tab is active, showing a list of available tools on the left: "search_trials" (highlighted), "get_trial", and "count_trials_by_sponsor".

The main panel displays the configuration for the "search_trials" tool. It includes a search bar with the text "Search clinical trials by title. limit is 1-50." and a toggle for "Paginated" results. Below this, there is a "Query" input field containing the word "Diabetes", a "Limit" input field set to "10", and an "Execute Tool" button. An "Edit as JSON" toggle is also present.

Continue where you left off: Chrome can restore your tabs every time you restart. To turn this on, go to [Settings](#).

trials Servers Tools Prompts Resources

Tools

Search tools...

Paginated

search_trials

get_trial

count_trials_by_sponsor

Results

```
1- {
2  "ok": true,
3  "data": [
4    {
5      "id": 6,
6      "trial_title": "Empagliflozin for Type 2 Diabetes #6",
7      "nct_number": "NCT10000005",
8      "available_slots": 10,
9      "sponsor_id": null
10   },
11   {
12     "id": 18,
13     "trial_title": "Empagliflozin for Type 2 Diabetes #18",
14     "nct_number": "NCT10000017",
15     "available_slots": 10,
16     "sponsor_id": null
17   },
18   {
19     "id": 25,
20     "trial_title": "Metformin for Type 2 Diabetes #25",
21     "nct_number": "NCT10000024",
22     "available_slots": 10,
23     "sponsor_id": null
24   },
25   {
26     "id": 35,
27     "trial_title": "Aspirin for Type 2 Diabetes #35",
28     "nct_number": "NCT10000034",
29     "available_slots": 10,
30     "sponsor_id": null
31   },
32   {
33     "id": 39,
34     "trial_title": "Lisinopril for Type 2 Diabetes #39",
35     "nct_number": "NCT10000038",
36     "available_slots": 10,
37     "sponsor_id": null
38   },
39   {
40     "id": 42,
41     "trial_title": "Lisinopril for Type 2 Diabetes #42",
42     "nct_number": "NCT10000041",
43     "available_slots": 10,
44     "sponsor_id": null
45   },
46   {
47     "id": 43,
48     "trial_title": "Semaglutide for Type 2 Diabetes #43",
49     "nct_number": "NCT10000042",
50     "available_slots": 10,
51     "sponsor_id": null
52   }
53 ]
54 }
```

Protocol

Messa

Newest

Expo

17:3

TOOL

17:2

TOOL

17:2

RESO

17:2

PROP

17:2

TOOL

17:2

Copyright ©

With invalid input:

search_trials

Search clinical trials by title. limit is 1-50.

Edit as JSON

Query *

Limit

5

Execute Tool

Results

```
1- {
2  "ok": false,
3  "data": null,
4  "error": "query must be a non-empty string"
5 }
```

Get Trials will pull the trial by adding the trial id

The image displays two screenshots of a web application interface for 'trials'. The top screenshot shows the 'get_trial' tool being executed with the trial ID '5001'. The tool's description is 'Get one trial (with its sponsor) by id.' The 'Trial id' input field contains '5001', and the 'Execute Tool' button is visible. The right sidebar shows a 'Messages' panel with a 'LEGACY' tab and a 'Newest First' dropdown. The bottom screenshot shows the same interface after an invalid input. The 'Results' panel displays a JSON error response:

```
1 {
2   "ok": true,
3   "data": {
4     "id": 5001,
5     "trial_title": "Pflizer Diabetes Phase III",
6     "rect_number": "K199999081",
7     "available_slots": 25,
8     "sponsor": {
9       "id": 1,
10      "sponsor_name": "Pflizer Research"
11    }
12   },
13   "error": null
14 }
```

 The 'Messages' panel on the right shows a 'LEGACY' tab and a 'Clear' button. The 'Messages' list shows a 'get' message at 17:31:43 and a 'sea' message at 17:30:01.

With invalid input:

get_trial

Get one trial (with its sponsor) by id.

Trial Id *

0

Execute Tool

Tools

Search tools...

Paginated

search_trials

get_trial

count_trials_by_sponsor

Results

```
1 {
2   "ok": false,
3   "data": null,
4   "error": "trial_id must be a positive integer"
5 }
```

Count trials by sponsor id will give the aggregate count of trials and total available slots for sponsor

trials

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search_trials

get_trial

count_trials_by_sponsor

count_trials_by_sponsor

Aggregate: number of trials and total available slots for a sponsor.

Edit as JSON

Sponsor Id *

1

Execute Tool

×

Results

```
1 {
2   "ok": true,
3   "data": {
4     "sponsor_id": 1,
5     "sponsor_name": "Pfizer Research",
6     "trial_count": 1,
7     "total_available_slots": 25
8   },
9   "error": null
10 }
```

With invalid input

count_trials_by_sponsor

Aggregate: number of trials and total available slots for a sponsor.

Sponsor Id *

5001

Execute Tool

× Results

```
1 {  
2   "ok": false,  
3   "data": null,  
4   "error": "sponsor 5001 not found"  
5 }
```

Part 3 - Tool Contracts Under Stress:

- a. **Rejected calls from Part 2B:** All three tools return the same envelope, {ok, data, error}. Each rejected call below is the same one captured in the Part 2B Inspector screenshots. The first two calls are rejected by input validation (the input is malformed). The third passes validation and is rejected because the record does not exist. In all three cases the tool returns an error instead of raising an exception.

Tool	Expected input schema	Rejected JSON input	Returned error output	Why it was rejected
search_trials	{"query": string (non-empty), "limit": integer 1-50}	{"query": "", "limit": 5}	{"ok": false, "data": null, "error": "query must be a non-empty string"}	The query is empty, so there is nothing to search for. The input check fails before any database call.
get_trial	{"trial_id": integer >= 1}	{"trial_id": 0}	{"ok": false, "data": null, "error": "trial_id must be a positive integer"}	Ids start at 1, so 0 can never match a trial.
count_trials_by_sponsor	{"sponsor_id": integer >= 1}	{"sponsor_id": 5001}	{ "ok": false, "data": null, "error": "sponsor 5001 not found" }	The input has the right shape, but no sponsor with id 5001 exists. This is a lookup failure, not a format failure.

b. Timeout and bounded exponential-backoff retry:

```

MAX_RETRIES = 3           # retries after the first attempt (4 attempts total)
BASE_DELAY = 0.05          # seconds; doubles each retry: 0.05, 0.1, 0.2
TIMEOUT = 2.0              # seconds per attempt

```

```

try:
    for attempt in range(max_retries + 1):
        attempts = attempt + 1
        try:
            future = pool.submit(fn, *args, **kwargs)
            result = future.result(timeout=timeout)
            return result, attempts
        except FutureTimeout:
            last_error = f"timed out after {timeout}s"
        except TransientError as e:
            last_error = str(e)
        except Exception as e:
            last_error = f"unexpected error: {e}"
        log.warning("attempt %d failed: %s", attempts, last_error)
        if attempt < max_retries:
            time.sleep(base_delay * (2 ** attempt))
    return err(f"failed after {attempts} attempts: {last_error}"), attempts

```

Each attempt has a 2-second timeout. A failed or timed-out attempt is retried up to 3 more times, waiting 0.05s, 0.1s, then 0.2s (the delay doubles each time). The retry count is bounded, so the call can never loop forever. If every attempt fails, the function returns `errpr` instead of raising an exception.

```

(venv) PS C:\Users\vivek\data260-5825\code\mcp> python part3_experiment.py 2>$null | Tee-Object
hw05\RUN_LOG_part3.txt
VERIFY_SEED = 265825

=== DEMO: success on the first attempt ===
outcomes script : [False] (True = injected failure)
attempts used   : 1
latency (ms)    : 29.8
result ok       : True
error           : None

=== DEMO: failure first, success after retry ===
outcomes script : [True, False] (True = injected failure)
attempts used   : 2
latency (ms)    : 54.4
result ok       : True
error           : None

=== DEMO: failure after all retries (clean error) ===
outcomes script : [True, True, True, True] (True = injected failure)
attempts used   : 4
latency (ms)    : 352.7
result ok       : False
error           : failed after 4 attempts: injected failure

| Injected failure rate | Success rate | Mean latency (ms) | p99 latency (ms) |
|---|---|---|---|
| 0% | 100.0% | 0.81 | 1.66 |
| 20% | 100.0% | 34.82 | 352.89 |
| 50% | 94.0% | 103.72 | 356.3 |

rows written: 150 (expected 150)

```

- Success on the first attempt. - Latency 29.8 ms with 1 attempt and result is true
- Failure on the first attempt followed by success after a retry- Latency 54.4 ms with 2 attempts and result is true
- Failure after all allowed retries, returning a clean error result instead of crashing- Latency 352.7 ms with 4 attempts and result is false as it failed.

c. Reproducible fault injection (VERIFY_SEED):

Each attempt draws one random number, and the attempt fails if the number is below the failure rate (0%, 20%, 50%).

```
class FlakyInjector:
    """Seeded failure injector. Same seed gives the same fail/succeed sequence."""

    def __init__(self, rate, seed):
        self.rate = rate
        self.rng = random.Random(seed)

    def wrap(self, fn):
        def inner(*args, **kwargs):
            if self.rng.random() < self.rate:
                raise TransientError("injected failure")
            return fn(*args, **kwargs)
        return inner

Rutuja Saste> cd code\mcp
Rutuja Saste> python part3_experiment.py 2>$null | Tee-Object -FilePath ../../reports/hw05\RUN_LOG_part3_run2.txt
VERIFY_SEED = 265825

=== DEMO: success on the first attempt ===
outcomes script : [False] (True = injected failure)
attempts used    : 1
latency (ms)     : 37.5
result ok        : True
error            : None

=== DEMO: failure first, success after retry ===
outcomes script : [True, False] (True = injected failure)
attempts used    : 2
latency (ms)     : 54.5
result ok        : True
error            : None

=== DEMO: failure after all retries (clean error) ===
outcomes script : [True, True, True, True] (True = injected failure)
attempts used    : 4
latency (ms)     : 352.8
result ok        : False
error            : failed after 4 attempts: injected failure

| Injected failure rate | Success rate | Mean latency (ms) | p99 latency (ms) |
|---|---|---|---|
| 0% | 100.0% | 0.99 | 3.1 |
| 20% | 100.0% | 35.4 | 355.13 |
| 50% | 94.0% | 103.93 | 355.86 |

rows written: 150 (expected 150)
```

The three retry demos and the 150-call fault-injection run, using VERIFY_SEED = 265825. Each call used a 2 s timeout and up to 3 retries with backoff of 0.05 s, 0.1 s and 0.2 s. All 150 calls are saved in reports/hw05/raw/fault_injection_150.csv.

Demo	Attempts used	Latency	Result
Success on the first attempt	1	37.5 ms	ok: True
Failure first, then success after a retry	2	54.5 ms	ok: True
Failure after all retries	4	352.8 ms	ok: False, clean error, no crash

Injected failure rate	Succesrate	Mean latency (ms)	p99 latency (ms)
0%	100.0%	0.99	3.10
20%	100.0%	35.40	355.13
50%	94.0%	103.93	355.86

From the results we can say that the retry policy suitable for an interactive assistant mostly as, with no injected failures (0%), calls finish in about 1 ms. With 20% failures, retrying recovered every call (100% success) and the mean latency was only 35.40 ms. The slowest calls (p99, about 355 ms) are the cost of waiting between retries: $0.05 + 0.1 + 0.2 = 0.35$ seconds. That is under half a second, so a user would barely notice. With 50% failures the policy is weaker. Success was 94%, so 3 of 50 calls still failed after all 4 attempts, and the mean latency rose to 103.93 ms because many calls needed retries. Even so, those failures came back as a clean `{ok: false}` error and nothing crashed. If a real service failed half the time, the right fix would be to repair the service, not to add more retries.

How you would modify the number of retries, timeout, or backoff delays for batch processing, where longer processing times are acceptable?

Batch jobs can wait longer, so success matters more than speed:

Retries: raise from 3 to about 8, so almost no call fails permanently.

Timeout: raise from 2 seconds to 10 to 30 seconds, so slow but healthy calls are not cut off.

Backoff: start with a longer wait (0.5 to 1 second), double it each time up to a cap of about 30 seconds, and add a small random delay so many jobs do not retry at the same moment.

Part 4 - One Safe Tool Entry Point and Offline Tests:

9 tests across the three domain tools (valid input, invalid input, unknown tool, bad arguments, envelope shape). They run offline against a fake database, with no MySQL, no network and no LLM. Result: 9/9 PASS. `execute_tool` is the single entry point for every tool call. It looks up the tool by name, validates the inputs, and always returns a JSON string in the `{ok, data, error}` shape without raising.

```
Rutuja Saste> python test_runner.py
PASS t_search_valid
PASS t_search_empty_query
PASS t_search_bad_limit
PASS t_get_valid
PASS t_get_invalid_id
PASS t_count_invalid_sponsor_type
PASS t_unknown_tool
PASS t_bad_arguments
PASS t_envelope_shape

9/9 tests passed
Rutuja Saste> █
```

I. Safety rule:

I. Safety rule:

```
0 MAX_AGENT_LIMIT = 25 # safety rule: an agent may not pull more than 25 records
1
2
3 def check_safety(name, inputs):
4     """Return an error message if the call breaks the safety rule, else None."""
5     if name == "search_trials" and isinstance(inputs, dict):
6         limit = inputs.get("limit", 10)
7         if isinstance(limit, int) and limit > MAX_AGENT_LIMIT:
8             return f"safety rule: search_trials limit {limit} exceeds "
9                 f"the maximum of {MAX_AGENT_LIMIT} records per call"
10    return None
```

Agent loop

```
def run_agent(user_input, model_name, db_name, max_steps=MAX_STEPS, log_path=LOG_FILE):
    model = model or ollama_model(db)

    messages = [
        ("role": "system", "content": "You answer questions about clinical trials. "
        "Use the tools to get real data. Answer briefly."),
        ("role": "user", "content": user_input),
    ]

    tool_calls_made = 0
    blocked = False
    run_id = int(time.time() * 1000)
    for step in range(1, max_steps + 1):
        resp = model.chat(messages)
        msg = _get(resp, "message")
        calls = _get(msg, "tool_calls") or []
        content = _get(msg, "content") or ""
        if not calls:
            log(("run": run_id, "step": step, "event": "final",
            "answer": content, "stop_reason": "completed", log_path)
            return ("answer": content, "steps": step, "stop_reason": "completed",
            "tool_calls": tool_calls_made, "safety_blocked": blocked)
        messages.append(("role": "assistant", "content": content))
        for c in calls:
            fn = _get(c, "function")
            name = _get(fn, "name")
            args = dict(_get(fn, "arguments") or {})
            result = executor.execute_tool(name, args, db=db)
            tool_calls_made += 1
            if "safety rule" in result:
                blocked = True
            log(("run": run_id, "step": step, "event": "tool call", "tool": name,
            "input": args, "result": json.loads(result), log_path)
            messages.append(("role": "tool", "content": result, "name": name))
    log(("run": run_id, "step": max_steps, "event": "stop",
    "stop_reason": "max_steps", log_path)
```

The loop counts steps, stops at max_steps (5), and logs every tool call, input, result and the final stop reason to agent_runs.jsonl

III. Additional tests

Part 4 and Part 5 offline test runner: 11 tests, all PASS. They run without a database, network or LLM.

```
Rutuja Saste> python test_runner.py
PASS t_search_valid
PASS t_search_empty_query
PASS t_search_bad_limit
PASS t_get_valid
PASS t_get_invalid_id
PASS t_count_invalid_sponsor_type
PASS t_unknown_tool
PASS t_bad_arguments
PASS t_envelope_shape
PASS t_safety_blocks
PASS t_agent_max_steps
```

11/11 tests passed

Rutuja Saste> █

Code:

```
def execute_tool(name, inputs, db=None):
    try:
        if name not in TOOLS:
            return json.dumps(core.err(f"unknown tool: {name}"))
        if not isinstance(inputs, dict):
            return json.dumps(core.err("inputs must be an object"))
        violation = check_safety(name, inputs)
        if violation:
            return json.dumps(core.err(violation))
        result = TOOLS[name](**inputs, db=db)
        return json.dumps(result)
    except TypeError as e:
        return json.dumps(core.err(f"bad arguments: {e}"))
    except Exception as e:
        return json.dumps(core.err(f"internal error: {e}"))
```

IV. Metrics:

```
INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
{"question": "How many trials does sponsor 1 have, and how many slots in total?", "steps": 2, "stop_reason": "completed", "tool_calls": 1, "safety_blocked": false}
INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
{"question": "Find trials about Diabetes and tell me the first one's NCT number.", "steps": 2, "stop_reason": "completed", "tool_calls": 1, "safety_blocked": false}
INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
{"question": "Look up trial 5001 and name its sponsor.", "steps": 2, "stop_reason": "completed", "tool_calls": 1, "safety_blocked": false}
INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
{"question": "Give me 100 trials about Metformin.", "steps": 4, "stop_reason": "completed", "tool_calls": 3, "safety_blocked": true}
```

Scenario	Steps	Stop reason	Tool calls	Safety blocked
How many trials does sponsor 1 have, and how many slots in total?	2	completed	1	False
Find trials about Diabetes and tell me the first one's NCT number.	2	completed	1	False
Look up trial 5001 and name its sponsor.	2	completed	1	False
Give me 100 trials about Metformin.	4	completed	3	True

Reflection:

I chose scenario 4, "Give me 100 trials about Metformin," because it shows the safety rule working inside the agent loop (run 1791225521181).

Step 1: the model asked for `search_trials` with `limit: 100`. The harness sent this through `execute_tool`, and the safety rule (maximum 25 records per call) blocked it. It returned `{ok: false, data: null, error: "safety rule: ..."}` without raising an exception.

Step 2: the model read the error and retried with `limit: 25`. This call was allowed and returned 25 Metformin trials.

Step 3: the model repeated the same 25-record call with the arguments in a different order. This was redundant. It did not need new data, but the harness has no rule against repeated calls, so it ran and counted as a third tool call.

Step 4: the model gave its final answer, so the harness stopped with reason completed. It did not stop because of `max_steps`, since the limit is 5 and only 4 steps were used.

The run shows two things. The safety rule worked: a blocked call is returned as ordinary data, and the model recovered from it. It also shows a weakness. The model said "24 trials" but the log shows 25 rows were returned, so a model's summary should not be trusted without checking the tool result. The harness also wasted one call on a duplicate. A future improvement would be to detect repeated identical calls and stop them.

Self-check: Verification.json:

```
Rutuja Saste> python code\verify_hw5.py | Tee-Object -FilePath reports\hw05\RUN_LOG.txt
PASS FastAPI backend responds on port 8425
PASS trials MCP server lists 3 tools and answers a call ['search_trials', 'get_trial', 'count_trials_by_sponsor']
PASS meals MCP server lists 4 tools and answers a call ['search_meals_by_name', 'meals_by_ingredient', 'random_meal', 'meal_details']
PASS offline tests all pass ['11/11 tests passed']
PASS 150 fault-injection records saved 150
PASS agent_runs.jsonl exists
all passed: True
```

AI_USE.md

1. What I used an AI assistant for, and what I did myself

I used AI to explain the assignment, understand and write the code for Redux slice, MCP servers, retry logic and fix errors. I did all the validations against MCP server. Updated the SQL tables by including the sponsor table and linking it with the trials table. Did overall changes for the flow and also ran the verification tests.

2. One AI-produced output that was wrong or unsuitable

The AI gave me MCP server code using `FastMCP`, but `pip install "mcp[cli]"` installed MCP 2.x, where that class was renamed, so `mcp dev` failed with `No module named 'mcp.server.fastmcp'`. Other AI problems I hit: a stray period in AI-written comments in `seed_hw4.py` caused a `SyntaxError` on my second laptop; `jinja2` was missing from the AI's install list; and in scenario 4 the agent said "24 trials" while the tool result had 25 rows.

3. How I detected the problem

The MCP error message named the rename directly, and I confirmed the installed version with `pip show mcp` (2.3.0). The other issues showed up as errors when I ran the code on a clean machine, and I counted the rows in `agent_runs.jsonl` to catch the "24 vs 25" mistake.

4. What I changed and why it works now

I pinned `mcp<2` (1.30.0), which has `FastMCP`, and launched the Inspector with my venv Python instead of `uv`; both servers now connect. I removed the stray periods, installed `jinja2`, and set `DATABASE_URL`, so the backend and seed run on both laptops. For the agent, I documented that a model's summary must be checked against the tool result.